GHC.IO: https://hackage-content.haskell.org/package/base/docs/GHC-IO.html

GHC.IO

```
newtype IO a = IO (State# RealWorld -> (# State#
RealWorld, a #))
unIO :: IO a -> State# RealWorld -> (# State# RealWorld,
a #)

liftIO :: IO a -> State# RealWorld -> STret RealWorld a
mplusIO :: IO a -> IO a -> IO a
unsafePerformIO :: IO a -> a
unsafeInterleaveIO :: IO a -> IO a
unsafeDupablePerformIO :: IO a -> a
unsafeDupableInterleaveIO :: IO a -> IO a
noDuplicate :: IO ()

-- ST
stToIO :: ST RealWorld a -> IO a
ioToST :: IO a -> ST RealWorld a
unsafeIOToST :: IO a -> ST s a
unsafeSTToIO :: ST s a -> IO a

-- Error
catch :: Exception e => IO a -> (e -> IO a) -> IO a
catchException :: Exception e => IO a -> (e -> IO a) ->
IO a
catchAny :: IO a -> (forall e. (HasExceptionContext,
Exception e) => e -> IO a) -> IO a
throwIO :: (HasCallStack, Exception e) => e -> IO a
mask :: ((forall a. IO a -> IO a) -> IO b) -> IO b
mask_ :: IO a -> IO a
uninterruptibleMask :: ((forall a. IO a -> IO a) -> IO
b) -> IO b
uninterruptibleMask_ :: IO a -> IO a
data MaskingState
    = Unmasked
    | MaskedInterruptible
    | MaskedUninterruptible

getMaskingState :: IO MaskingState
unsafeUnmask :: IO a -> IO a
interruptible :: IO a -> IO a
onException :: IO a -> IO b -> IO a
bracket :: IO a -> (a -> IO b) -> (a -> IO c) -> IO c
finally :: IO a -> IO b -> IO a
evaluate :: a -> IO a
mkUserError :: [Char] -> SomeException
```

RealWorld
Control.Monad.ST: ttps://hackage-content.haskell.org/package/base/docs/Control-Monad-ST.html#t:RealWorld

==========
======

```
This
is
code
Monaco 14
```